

Université Claude Bernard Lyon 1

Licence 2 Informatique — LIFAPCD : Programmation et Conception

Grand Line Legends

RPG tour par tour — thème One Piece

C++11

SDL2

MVC

Doxygen

WinTXT

Dépôt : forge.univ-lyon1.fr/p2405025/grand-line-legends

Binaires : `bin/GLL` (SDL2) · `bin/GLL_text` (terminal)

Tests : 33 tests de régression — tous passent

Semestre Printemps 2025/2026 — Avril 2026

1. Sommaire

1. Introduction et présentation du projet

1. Contexte et objectifs
2. Contraintes techniques imposées

2. Architecture MVC

1. Organisation en couches
2. Flux de données et événements

3. Description du modèle

1. Personnages, capacités et effets
2. Joueur et Bank
3. Campagne : 18 arcs narratifs
4. Grille de combat
5. Shop et Coffres
6. Familles et bonus d'équipe
7. Sauvegarde et paramètres

4. Description du contrôleur

1. Game : contrôleur central
2. Battle : moteur de combat

5. Description de la vue

1. ViewText et librairie WinTXT
2. ViewGraph et SDL2

6. Format des données

7. Tests de régression

8. Bilan et conclusion

2. Introduction et présentation du projet

2.1 Contexte et objectifs

Grand Line Legends est un jeu de rôle (RPG) tour par tour développé dans le cadre du cours LIFAPCD à l'Université Claude Bernard Lyon 1. Il s'inspire de l'univers de *One Piece* et de mécaniques de jeux comme *Raid Shadow Legends* : collection de personnages via des fragments, progression dans une campagne en arcs narratifs, et système de boutique à offres temporisées.

Le projet comprend **67 personnages jouables** issus de la licence *One Piece*, chacun doté de statistiques propres (PV, vitesse, attaque, défense, Haki Royauté/Armement/Observation), de capacités uniques et d'une rareté allant de Commun à Mythique.

Fonctionnalités principales :

- **Menu principal** → Jouer / Paramètres / Quitter
- **Bank** : gestion des 67 personnages, déblocage par fragments, amélioration par berries
- **Campagne** : 18 arcs (Romance Dawn → Panthéon), sbires + mini-boss + boss
- **Combat** : tour par tour sur grille 2×10, ordre par vitesse décroissante
- **Shop** : offres journalière / hebdomadaire / mensuelle + 5 coffres permanents
- **Familles** : bonus d'équipe si membres d'une même famille (Mugiwara, Empereurs...)
- **Sauvegarde automatique** à la fermeture

2.2 Contraintes techniques imposées

Le sujet impose un ensemble de règles strictes de programmation que le projet respecte intégralement :

Règle	Application dans le projet
Pas de variable globale	Tout est transmis par constructeur ou référence
Passage par <code>const Type&</code>	Tous les paramètres en entrée lecture-seule
<code>const</code> sur les accesseurs	<code>int getPV() const;</code> sur 100% des getters

Règle	Application dans le projet
<code>assert()</code> sur préconditions	Validations en début de méthode critique
Chaque <code>new</code> a son <code>delete</code>	Vérifié par Valgrind, destructeurs explicites
Doxygen sur tous les headers	<code>/** @brief ... @param ... @return ... */</code>
Pas de <code>cout</code> / <code>cin</code> dans la vue texte	WinTXT utilisé exclusivement
Version terminale + version SDL2	<code>make text</code> et <code>make</code>
Tests de régression	33 tests avec <code>assert</code> dans <code>tests/</code>

3. Architecture MVC

3.1 Organisation en couches

Le projet adopte une architecture **Modèle–Vue–Contrôleur (MVC)** stricte. Chaque couche a un rôle exclusif et les dépendances sont orientées dans un seul sens : la Vue ne modifie jamais le Modèle directement.

Couche	Rôle	Fichiers principaux
Modèle	Données du jeu : personnages, campagne, combat, boutique, sauvegarde. Zéro affichage.	<code>src/model/</code> — 16 modules
Contrôleur	Logique de jeu : gestion des états, dispatch des événements, moteur de combat.	<code>src/controller/</code> — Game, Battle, Event
Vue	Affichage uniquement. Lit l'état du contrôleur, émet des événements. Ne touche pas au modèle.	<code>src/view/</code> — ViewText, ViewGraph, WinTXT

Le schéma suivant résume les relations entre les modules principaux :



3.2 Flux de données et événements

La vue communique avec le contrôleur uniquement via le système d'événements (`EventType`). Le cycle complet à chaque frame est le suivant :

1. La vue capture une touche clavier (WinTXT ou SDL_Event)
2. Elle appelle `game.input(touche)` qui retourne un `EventType`
3. Elle construit un `Event{type}` et appelle `game.update(&ev)`
4. `Game::update()` modifie le modèle selon le type d'événement
5. La vue appelle `render()` et lit l'état du jeu en lecture seule via les accesseurs

Avantage de cette architecture : les deux vues (terminale et graphique) sont parfaitement interchangeables. Elles partagent exactement le même contrôleur `Game` et le même modèle — seul l'affichage diffère.

4. Description du modèle

4.1 Personnages, capacités et effets

Character

Classe centrale du modèle. Elle encapsule toutes les statistiques d'un personnage : points de vie (`pv`), vitesse, attaque, défense, les trois niveaux de Haki (Royauté, Armement, Observation), le niveau actuel, et un tableau de pointeurs vers ses capacités. Elle est chargée depuis `data/Characters.csv` par le constructeur de `Player`.

Le coût d'amélioration est calculé par `upgradeCost()` selon le niveau et la rareté du personnage. La méthode `updateLvl(delta)` augmente le niveau et met à jour les statistiques en conséquence. La méthode `resetCombatState()` réinitialise tous les effets temporaires entre deux combats (saignement, étourdissement, résistance).

Capacity

Représente une capacité d'un personnage. Chaque capacité est définie par son type d'effet (`Damage` , `Heal` , `Stun` , `Bleeding` ...), son ciblage (`Self` , `Ally` , `Enemy` , `AllEnemies` ...), sa valeur de dégâts ou de soin, et son `percentage` d'occurrence. Les capacités passives ont `percentage = 100`.

Effect

Décrit l'effet appliqué par une capacité : type (`EffectType`), valeur numérique et durée en tours. Les effets sont appliqués par `Battle::playCharacter()` lors du résolution de chaque attaque.

Effets disponibles

Effet	Description
Damage	Inflige des dégâts à la cible
Heal	Restaure des PV à la cible
Stun	Empêche la cible d'agir ce tour
Bleeding	Dégâts sur la durée (DOT)

Effet	Description
Resist	Réduit les dégâts reçus
Push / Pull	Déplace la cible sur la grille
Swap	Échange de position entre deux personnages

4.2 Joueur et Bank

La classe `Player` gère l'intégralité de l'état du joueur :

- **Bank** : tableau de 67 pointeurs `Character*` , chargé depuis le CSV au démarrage
- **Équipe** : sous-ensemble de la bank (1 slot au départ, +1 par niveau de compte, max 10)
- **Ressources** : berries (monnaie), XP (progression de compte de niveau 1 à 10)
- **Fragments** : tableau de structures `FragEntry{charName, count}` . 100 fragments = déblocage d'un personnage
- **Personnages débloqués** : liste des noms débloqués, consultée par `isUnlocked(name)`

Système de fragments : chaque victoire en campagne fait tomber des fragments d'un personnage spécifique avec une probabilité définie par arc. La méthode `addFragments(name, n)` incrémente le compteur, et `unlockCharacter(c)` vérifie qu'on atteint 100 avant de valider le déblocage.

4.3 Campagne : 18 arcs narratifs

La classe `Campaign` contient un tableau fixe de 18 pointeurs `Arc*`, construits dans son constructeur. La navigation se fait via `arcUp()` / `arcDown()` et `episodeUp()` / `episodeDown()`.

Les 18 arcs de la campagne

Arc	Boss	Personnage débloqué	Personnage banni
Romance Dawn	Morgan	Zoro (100%)	—
Orange Town	Baggy	Nami (50%)	—
Village Sirop	Kuro	Usopp (100%)	—
Baratie	Don Krieg	Sanji (100%)	—
Arlong Park	Arlong	Nami (50%)	—
Logue Town	Smoker	Baggy (100%)	—
Whiskey Peak	Mr.5 & Miss Valentine	Vivi (100%)	—
Little Garden	Mr.3 & Miss GW	Brogy (50%)	—
Drum Island	Wapol	Chopper (100%)	Nami
Alabasta	Crocodile	Robin (100%)	—
Skypea	Ener	—	—
Water Seven	Franky	Franky (50%)	—
Enies Lobby	Rob Lucci	Franky (50%)	—
Thriller Bark	Gecko Moria	Brook (100%)	—
Amazon Lily	Boa Hancock	Boa Hancock (100%)	—
Impel Down	Magellan	Bon Clay + multi (25%)	—
Marine Ford	Garp / Sengoku / Akainu	Ace (50%) + multi	—
Panthéon	Tous les Boss	—	—

Chaque arc contient plusieurs `Episode`, dont un mini-boss et un boss final. La difficulté de l'épisode (1 à 5) détermine le nombre d'ennemis et leur puissance. À la victoire, la méthode

`Game::applyRewards()` tire aléatoirement les récompenses selon leurs probabilités de drop.

9 / 24

4.4 Grille de combat

Le combat se déroule sur une grille 2×10 représentée par la classe `BattleMap`, composée de 20 cellules `Squares`. La ligne 0 accueille l'équipe du joueur, la ligne 1 l'équipe ennemie.

- Chaque `Squares` mémorise l'identifiant du personnage qui l'occupe
- Les effets **Push** et **Pull** déplacent un personnage sur la grille
- L'effet **Swap** échange deux positions sur la grille

4.5 Shop et Coffres

La classe `Shop` génère automatiquement ses offres à l'instanciation en utilisant l'heure système (`time()` → `struct tm`). Les offres changent réellement selon le calendrier :

- **Offre journalière** : change selon `tm_wday` (jour de la semaine)
- **Offre hebdomadaire** : change selon `(tm_mday-1)/7` (semaine du mois)
- **Offre mensuelle** : change selon `tm_mon`
- **5 permanentes** : Coffre Rare, Coffre Super Rare, Coffre Épique, Coffre Légendaire, Coffre Mythique

La classe `Coffre` simule une loot box : sa méthode `open()` retourne un pointeur `Reward*` (alloué dynamiquement, à libérer par l'appelant) contenant des fragments d'un personnage aléatoire correspondant à la rareté du coffre.

4.6 Familles et bonus d'équipe

Le jeu implémente 8 familles de personnages. Lorsque tous les membres d'une famille sont présents dans l'équipe active, un bonus est appliqué en combat.

Famille	Membres	Bonus
Monkey D.	Garp, Luffy, Dragon	+25% PV
Trio de Frères	Sabo, Ace, Luffy	+25% Attaque
Révolutionnaires	Sabo, Dragon, Ivankov	+15% Défense
Empereurs	BarbeBlanche, BarbeNoire, Shanks, Kaido, BigMom	+20% PV

Famille	Membres	Bonus
Amiraux	Akainu, Aokiji, Kizaru, Fujitora, Ryokugyu	+20% Défense
Mugiwara	Les 9 Chapeaux de Paille	+20% Attaque
Marines	Garp, Sengoku, Smoker, Koby	+15% Défense
Équipage BarbeBlanche	BarbeBlanche, Ace, Marco, Izo	+20% PV

4.7 Sauvegarde et paramètres

La classe statique `SaveData` gère la persistance via un fichier texte `data/save.txt`. À la fermeture, `Game::save()` sérialise : le pseudo, le niveau de compte, l'XP, les berries, la taille d'équipe maximale, la liste des personnages débloqués avec leurs fragments, et la progression dans la campagne.

Au lancement, la vue propose de charger la sauvegarde existante ou de démarrer une nouvelle partie. Trois emplacements de sauvegarde sont disponibles (slots 1 à 3).

La classe `Settings` persiste quant à elle dans `data/settings.txt` les préférences fps (30/60), volume (0-100) et luminosité (0-100).

5. Description du contrôleur

5.1 Game : contrôleur central

`Game` est le pivot de l'application. Il possède toutes les ressources du jeu (`Player*` , `Campaign*` , `Settings*` , `Shop*` , `Battle*`) et les libère dans son destructeur.

Machine à états (GameState)

L'état courant du jeu est stocké dans `state : GameState` . Chaque état correspond à un écran et à une méthode d'entrée dédiée :

État	Écran affiché	Méthode d'entrée
MAIN_MENU	Menu principal (J / P / Q)	inputMainMenu()
SETTINGS	Paramètres fps/volume/luminosité	inputSettings()
PLAY_MENU	Sous-menu (Bank / Campagne / Shop)	inputPlayMenu()
BANK	Liste des 67 personnages	inputBank()
BANK_DETAIL	Fiche détaillée d'un personnage	inputBankDetail()
CAMPAIGN	Sélection d'arc	inputCampaign()
CAMPAIGN_ARC	Épisodes d'un arc	inputCampaignArc()
BATTLE_PREPA	Préparation du combat	inputBattlePrepa()
BATTLE	Résolution du combat	inputBattle()
TEAM / TEAM_CHANGE	Gestion de l'équipe	inputTeam/Change()
SHOP	Boutique	inputShop()
QUIT	Sauvegarde + fermeture	—

Construction de l'équipe ennemie

`buildEnemyTeam(enemy, difficulty)` sélectionne aléatoirement des personnages depuis la bank

du joueur. Le nombre d'ennemis croît avec la difficulté (1 à 5). Les personnages bannis par l'arc courant sont exclus de la sélection. Le niveau de difficulté est appliqué aux personnages ennemis via `updateLvl()` .

5.2 Battle : moteur de combat

`Battle` reçoit un `Player*` (joueur) et crée lui-même un `Player*` ennemi qu'il possède et détruit dans son destructeur. La `BattleMap` est également créée et gérée par `Battle` .

Déroulement d'un tour

1. `getOrder()` collecte tous les personnages vivants et les trie par vitesse décroissante
2. Pour chaque personnage dans l'ordre, `playCharacter(c)` est appelé
3. Si le personnage est *stunné* ou mort, son tour est sauté
4. Une capacité est choisie aléatoirement selon les `percentage` de chaque capacité
5. L'effet est appliqué aux cibles (gestion du ciblage multiple : `AllEnemies`, `AllAllies...`)
6. L'action est enregistrée dans le log (`BattleLog[64]`)

Fin de combat

`getWinner()` vérifie après chaque tour si tous les personnages d'un camp sont morts (`isDead(player)`). Il retourne 0 (victoire joueur), 1 (victoire ennemi) ou -1 (égalité). En cas de victoire, `Game::applyRewards()` distribue les récompenses.

Journal de combat : chaque action est stockée dans un tableau de `BattleLog` (attaquant, nom de la capacité, cible, valeur). La vue lit ce log à chaque rendu pour afficher le détail des échanges.

6. Description de la vue

6.1 ViewText et librairie WinTXT

`ViewText` est la vue terminale du jeu. Elle utilise **exclusivement** la librairie `WinTXT` fournie — aucun `cout`, `cin` ou `printf` n'est utilisé dans cette classe.

WinTXT

La librairie `WinTXT` (fournie, non modifiable) propose une interface de fenêtre texte basée sur les séquences ANSI. Elle expose :

- `print(char*)` — affiche une chaîne à la position courante
- `move(x, y)` — positionne le curseur
- `clear()` — efface l'écran
- `refresh()` — rafraîchit l'affichage
- `getKey()` — lit une touche non-bloquant

Mécanisme d'affichage

Pour convertir les `std::string` en `char*` requis par WinTXT, `ViewText` maintient un buffer intermédiaire `char buf[WIN_W+1]` copié caractère par caractère avant chaque appel à `win.print()`. La méthode `readString(x, y, maxLen)` lit les caractères un par un via `win.getKey()` pour construire le pseudo du joueur sans jamais utiliser `cin`.

Écrans implémentés

Chaque état du jeu possède sa méthode de rendu dédiée : `renderMainMenu()`, `renderSettings()`, `renderPlayMenu()`, `renderBank()`, `renderBankDetail()`, `renderCampaign()`, `renderCampaignArc()`, `renderBattlePrepa()`, `renderBattle()`, `renderTeam()`, `renderTeamChange()`, `renderShop()`, `renderQuit()`.

6.2 ViewGraph et SDL2

`ViewGraph` est la version graphique du jeu basée sur **SDL2** et **SDL2_ttf**. Elle implémente exactement

les mêmes écrans que `ViewText` mais avec du rendu 2D accéléré. La fenêtre est à taille fixe ; `SDL_RenderSetLogicalSize` gère le HiDPI et Wayland.

- `SDL_Window*` : fenêtre principale
- `SDL_Renderer*` : renderer accéléré
- `TTF_Font*` : police pour le rendu du texte

`handleSDLEvent(SDL_Event&)` traduit les touches SDL en caractères compatibles avec `Game::input(char)`, assurant la même logique de contrôle que la version texte.

Compilation : la version SDL2 nécessite les bibliothèques `libsdl2-dev` et `libsdl2-ttf-dev` (Linux) ou `sdl2` + `sdl2_ttf` via Homebrew (macOS). Le Makefile les détecte automatiquement via `sdl2-config` ou `pkg-config`.

7. Format des données

7.1 Formats CSV et sauvegarde

data/Characters.csv

Contient les 67 personnages jouables. Chaque ligne définit les statistiques de base d'un personnage :

```
id,name,type,rarity,pv,speed,hakiR,hakiA,hakiO
1,Luffy,atk,c,4000,105,80,90,90
2,Zoro,atk,sr,3800,95,70,85,0
...
```

Champ	Valeurs possibles	Description
type	atk / def / sup / int / snp / mag	Rôle du personnage
rarity	c / r / sr / e / l / m	Commun → Mythique
hakiR/A/O	0–100	Haki Royauté / Armement / Observation

data/capacities/<Nom>.csv

Un fichier CSV par personnage. Exemple pour Ace :

```
id,character,name,type1,type2,damage,heal,targetType,targetCount,gearLevel,unlockLv
1,Ace,Fire Fist,fire,punch,2300,0,Enemy,1,3,15,Damage,40
2,Ace,Firewall,fire,,0,0,AllEnemies,0,2,8,Bleeding,30
3,Ace,Brothers Bond,,,0,1500,Ally,1,1,1,Heal,30
```

Champ	Description
targetType	Self / Ally / Enemy / AllAllies / AllEnemies / All
effect	Damage / Heal / Buff / Debuff / Resist / Stun / Swap / Bleeding / Push / Pull
percentage	Probabilité d'utilisation (100 = capacité passive)

data/save.txt

Format texte lisible, écrit par `SaveData::save()` :

```
Joueur      ← pseudo
1           ← accountLvl
0           ← xp
5000        ← berries
3           ← teamMax
2           ← nbUnlocked
Luffy 45    ← nom + fragments
Zoro 100    ← nom + fragments
0 2         ← arcIdx epIdx
```

17 / 24

8. Tests de régression

Le projet comprend **33 tests de régression** répartis en 4 binaires, tous basés sur des appels à `assert()`. Ils couvrent les modules critiques du modèle et du contrôleur.

8.1 Résultats

```
$ make tests

=== Tests Character ===
[OK] Character : constructeur par défaut
[OK] Character : updateLvl
[OK] Character : upgradeCost
[OK] Character : resetPV
[OK] Character : addToCapa
[OK] Character : tests de type (défaut atk)
[OK] Character : resetCombatState
=== Tous les tests Character passés ===

=== Tests Player ===
[OK] Player : constructeur
[OK] Player : addToBank / getBankCharacter
[OK] Player : addToUnlocked / isUnlocked
[OK] Player : addToTeam / removeFromTeam
[OK] Player : addTeamSize
[OK] Player : berries
[OK] Player : fragments
[OK] Player : addXP / montée de niveau
[OK] Player : changePseudo
[OK] Player : unlockCharacter via fragments
[OK] Player : addXP avec montées de niveau multiples
=== Tous les tests Player passés ===

=== Tests Battle ===
[OK] Battle : construction
[OK] Battle : getOrder (équipes vides)
[OK] Battle : isDead (équipe vide)
[OK] Battle : log
[OK] Battle : getWinner (égalité)
=== Tous les tests Battle passés ===
```

```
=== Tests Campaign ===  
[OK] Campaign : nombre d'arcs = 18  
[OK] Campaign : noms des arcs  
[OK] Campaign : noms des boss  
[OK] Campaign : structure des épisodes  
[OK] Campaign : navigation arcs  
[OK] Campaign : navigation épisodes  
[OK] Campaign : personnages bannis  
[OK] Campaign : chances de déblocage  
[OK] Campaign : complétion épisode  
[OK] SaveData : save / load  
=== Tous les tests Campaign passés ===
```

8.2 Détail des scénarios testés

Module	Scénario	Vérification
Character	Constructeur par défaut	Valeurs initiales cohérentes
	updateLvl(+1)	Niveau incrémenté correctement
	upgradeCost()	Coût > 0 et croissant avec le niveau
	resetPV()	PV remis à pvBase
	addToCapa()	nbCapa incrémenté, pointeur stocké
	Type string	Conversion CharType → string correcte
	resetCombatState()	isBleeding, stunned, resistValue réinitialisés
Player	Constructeur	Pseudo, berries, slots initiaux
	addToBank()	getNbBank() +1
	unlockCharacter()	isUnlocked() retourne true
	addToTeam()	getNbTeam() +1, getTeamMax() respecté
	addTeamSize(n)	teamMax augmenté de n
	addBerries()	Valeur correcte, pas de négatif absurde
	addFragments()	getFragments() retourne valeur accumulée
	addXP() → level up	accountLvl incrémenté, teamMax +1
	changePseudo()	getPseudo() retourne le nouveau nom
	unlockCharacter via fragments	isUnlocked() retourne true après 100 frags
	addXP avec montées multiples	accountLvl et teamMax corrects
Battle	Construction	Objet valide, aucun crash
	getOrder() équipes vides	Retourne tableau vide, pas de segfault
	isDead() équipe vide	Retourne true (pas d'ennemis = mort)
	addLog()	getNbLog() = 1, getLog(0) correct

Module	Scénario	Vérification
	getWinner()	-1 si les deux équipes vides
Campaign	Nombre d'arcs	getNbArcs() == 18
	Noms des arcs	Arc 0 = "Romance Dawn", Arc 17 = "Panthéon"
	Noms des boss	Vérification de 5 boss clés
	Structure épisodes	Chaque arc a au moins 3 épisodes
	Navigation arcs	arcUp/arcDown bornés à [0..17]
	Navigation épisodes	episodeUp/Down bornés
	Personnages bannis	isBanned("Nami") vrai dans Drum Island
	Chances déblocage	unlockChance ∈ [0.0, 1.0]
	Complétion épisode	ep.completed passe à true
	SaveData : save / load	Sauvegarde et rechargement sans perte de données

9. Bilan et conclusion

9.1 Ce qui a été réalisé

L'ensemble des fonctionnalités décrites dans le cahier des charges a été implémenté. Le projet est fonctionnel dans sa version terminale (`bin/GLL_text`) et la version graphique SDL2 est prête à être compilée sur un environnement disposant des bibliothèques SDL2.

Fonctionnalité	Statut
67 personnages chargés depuis CSV	✓ Complet
Capacités chargées depuis CSV (un fichier par perso)	✓ Complet
Bank : liste, détail, déblocage, amélioration	✓ Complet
Campagne : 18 arcs, épisodes, boss, récompenses	✓ Complet
Combat tour par tour sur grille 2×10	✓ Complet
Tous les effets (Stun, Bleeding, Push, Pull, Swap...)	✓ Complet
Familles et bonus d'équipe	✓ Complet
Shop avec offres temporisées (jour/semaine/mois)	✓ Complet
Coffres par rareté (5 niveaux permanents)	✓ Complet
Sauvegarde / chargement automatique (3 slots)	✓ Complet
Paramètres (fps, volume, luminosité)	✓ Complet
Vue terminale (WinTXT, sans cout/cin)	✓ Complet
Vue graphique SDL2 (fenêtre taille fixe)	✓ Complet (SDL2 requis)
Tests de régression (33 tests)	✓ 33/33 passent
Documentation Doxygen	✓ Tous les headers documentés
Makefile dual-target + docu + clean	✓ Complet

9.2 Points techniques notables

- **Absence totale de variable globale** : tout est transmis par constructeur ou référence. Le seul point d'entrée est `main()` qui instancie `Game` et la vue.
- **Gestion mémoire explicite** : chaque `new` est accompagné de son `delete` dans le destructeur du propriétaire. `Battle` est propriétaire de `enemy`, `Game` est propriétaire de toutes ses ressources, `Player` de ses personnages en bank.
- **Séparation stricte MVC** : la Vue ne modifie jamais le modèle, elle émet uniquement des `EventType` que `Game::update()` traite.
- **Shop temporisé réel** : les offres changent véritablement en fonction du jour, de la semaine et du mois calendaire — pas d'un compteur interne.

9.3 Commandes de compilation

```
# Version terminale
make text
./bin/GLL_text

# Version graphique (SDL2 requis)
sudo apt install libsdl2-dev libsdl2-ttf-dev
make
./bin/GLL

# Tests de régression
make tests

# Documentation Doxygen
sudo apt install doxygen graphviz
make docu
# → ouvre doc/html/index.html

# Nettoyage
make clean
```